# Codex Agent 系统架构与 Harness 机制报告

## 1 Codex 的作用与整体架构

Codex 是一个面向软件开发任务的 AI Agent 系统，通过模型推理、工具调用和任务状态管理完成代码相关工作。其整体架构采用分层设计，包括交互层、服务层和运行核心：CLI、IDE 等客户端负责用户交互；App Server 负责客户端与运行核心之间的通信；Core 则负责执行 Agent 任务，包括组织模型调用、管理上下文以及调度工具执行。

在 Core 内部，Codex Harness 负责控制 Agent 的执行流程，是实现智能体循环（Agent Loop）的核心机制。Harness 不直接替代模型进行决策，而是负责连接模型与外部环境：一方面向模型提供任务上下文和可用工具，另一方面接收模型产生的工具调用并控制实际执行过程。因此，模型负责生成下一步操作，而 Harness 负责维护执行状态并推动任务继续运行。

对于通过 App Server 接入的客户端，客户端与 Core 之间通过 JSON-RPC 协议交换请求和事件，使用户界面能够展示任务进度、处理权限审批以及恢复历史会话。

在任务组织方式上，Codex App Server 采用 Thread-Turn-Item 三层数据结构。其中，Thread 表示一个持续的会话，Turn 表示用户发起的一轮任务，Item 表示任务执行过程中的具体事件单元，例如用户消息、模型响应、工具调用以及执行结果。该设计使客户端能够追踪任务执行过程，并支持会话管理与恢复。

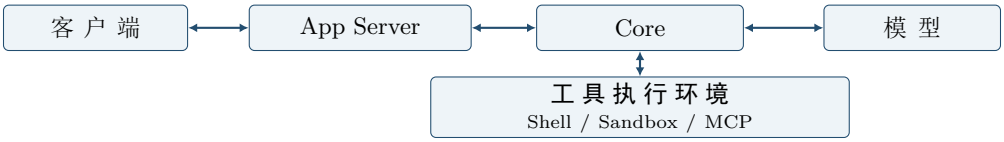



图 1: Codex 系统架构与组件通信图

## 2 Codex Harness 的核心运行机制与工程设计

### 2.1 Agent Loop：从任务请求到执行反馈

Codex Harness 将用户发起的一次任务组织为一个 Turn，并通过模型推理、工具执行和结果反馈形成循环。模型在每轮推理中根据当前上下文和执行结果动态决定下一步操作,Harness 决定这个动作如何被执行以及是否继续循环。一次完整的执行过程可以分为以下几个阶段。

**（1）任务上下文构建。**用户输入进入 Core 后，Harness 将其组织为模型请求。请求不仅包含当前用户消息，还结合已有会话历史、系统指令以及可用工具信息，使模型能够理解当前任务状态并生成下一步行动。

**（2）模型推理与工具调用。**模型根据当前上下文产生响应。若任务需要访问外部环境，模型会生成工具调用请求，例如读取文件、执行命令或修改代码。此时 Harness 负责解析模型输出，并将工具调用转交给对应执行模块。**模型负责决定“做什么”，Harness 负责控制“如何执行”。**

**（3）工具反馈驱动下一轮推理。**工具执行完成后，Harness 将执行结果重新加入任务上下文，并触发下一轮模型请求。例如，测试失败日志会成为下一轮推理的重要输入，使模型能够进一步定位问题、调整方案或继续执行。

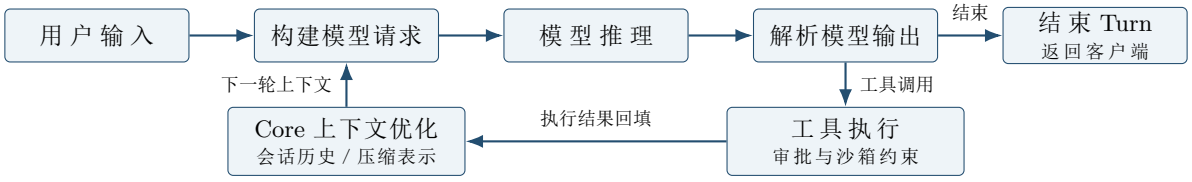



图 2: Harness 的 Agent Loop：模型决策、工具反馈与上下文更新形成闭环。

**（4）任务结束与 Turn 完成。**当模型生成最终响应，或当前任务不存在继续执行条件时，Harness 结束当前 Turn，并将结果返回客户端。

### 2.2 关键工程设计：上下文优化、工具执行与任务状态管理

Agent 系统与传统对话系统最大的区别在于，其任务执行过程通常持续较长时间，并且需要多次模型推理、工具调用以及环境交互。因此，Harness 的核心挑战是在保证任务完成质量的同时，控制推理成本、执行延迟以及任

务可靠性。Codex Harness 针对这些问题进行了多方面的工程设计。

**（1）上下文优化：在任务状态、推理质量与计算成本之间平衡**

对于长周期的软件开发任务，Agent Loop 会持续产生新的上下文，包括用户需求、模型响应、工具调用以及执行结果。如果简单累积全部历史，不仅会导致输入规模不断增加，还会增加模型推理成本，并降低上下文中有效信息的比例。因此，Codex Harness 的上下文优化并不是简单保存对话历史，而是围绕“如何为下一轮决策提供有效状态”进行设计。

在 Codex Core 中，一次 Turn 的执行过程会持续维护任务历史，并根据当前状态构造模型请求。`run_turn` 负责组织将模型产生的响应、工具调用以及工具执行结果加入会话历史，这些内容在后续请求中重新参与推理。

当历史内容持续增长时，Codex 采用 Compaction 机制对上下文进行压缩。Compaction 流程会读取当前历史记录，并通过专门的总结请求生成新的压缩表示，再使用新的摘要替换原始历史内容。这样既减少了上下文长度，又保留了任务执行过程中形成的重要信息。

在压缩过程中，Codex 重点处理的是**任务状态的连续性**。压缩后的上下文不能只是过去消息的摘要，还需要保留后续推理所依赖的信息。因此，Core 会重新组织压缩后的历史结构，并在摘要与最新用户请求之间插入必要的初始上下文，使模型仍然能够理解任务目标、环境约束以及当前执行阶段。这种设计避免了传统“滑动窗口”方式可能造成的任务背景丢失。

除了控制上下文规模，Codex 还针对多轮推理中的重复计算进行了优化。Agent 请求通常包含大量稳定内容，例如系统指令、工具定义以及项目级规则。如果每轮请求都重新处理这些内容，会产生额外的计算开销。因此，Codex 在构造 Prompt 时尽量保持稳定内容的位置和格式，使连续请求能够利用模型侧的 Prompt Cache，减少重复计算。

这种设计体现了 Codex 对 Agent 性能问题的系统考虑：上下文优化不仅关注“保留多少信息”，还关注“哪些信息值得保留”以及“如何降低重复计算”。Compaction 通过压缩历史控制上下文规模，Prompt Cache 通过复用稳定前缀降低推理成本，两者共同支撑 Agent 在长任务中的持续运行能力。

**（2）工具执行：模型决策与环境操作的解耦**

在 Agent 系统中，模型输出的本质是对下一步行动的规划，而不是实际执行能力。如果模型直接获得文件系统或 Shell 权限，会导致推理过程与环境操作高度耦合，难以控制执行风险，也难以扩展新的工具能力。因此，Codex Harness 将“决策”和“执行”进行分离：模型负责生成 Function Call，Harness 负责解析调用、选择执行路径并管理工具返回结果。

在 Codex Core 中，工具系统通过统一接口组织不同类型的外部能力。模型看到的是经过定义的工具描述（Tool Schema），而不是具体实现细节；当模型产生工具调用请求后，Core 根据工具名称和参数将请求分发到对应处理模块，例如 Shell 命令执行、文件操作或 MCP 工具。工具执行完成后，结果会重新进入会话上下文，成为下一轮模型推理的输入。

这种设计带来的第一个优势是**执行环境与模型能力解耦**。模型无需知道具体工具如何实现，只需要学习如何选择合适的工具；而 Harness 可以在工具执行阶段加入权限审批、Sandbox 限制以及错误处理机制。例如，对于代码修改或命令执行等具有副作用的操作，Harness 可以在真正执行前进行安全检查，而不会将控制权完全交给模型。

第二个优势是**工具生态的可扩展性**。由于 Agent Loop 只依赖统一的工具调用接口，新增工具不需要修改模型推理流程。例如，当系统增加 MCP 服务或新的开发工具时，只需要扩展对应工具实现并注册到 Harness 中，模型即可通过新的 Tool Schema 使用该能力。这种设计使 Agent 能够从单纯的代码生成器扩展为可以操作真实开发环境的工程助手。

因此，Codex 的工具架构并不是简单增加模型的操作权限，而是在模型推理和外部环境之间建立了一层可控的执行抽象，使 Agent 在获得环境交互能力的同时保持安全性、可扩展性和可维护性。

**（3）任务状态管理：支持持续执行与任务恢复**

Codex 通过 Thread-Turn-Item 结构化组织 Agent 执行状态，将用户消息、模型响应、工具调用以及执行结果作为独立事件进行管理。

这种设计使 Harness 能够追踪任务执行过程，并支持客户端实时展示状态变化。在任务中断或暂停后，系统也可以基于已有会话状态恢复执行，而无需重新开始整个流程。

## 3 优化思路

**上下文优化。**在压缩摘要之外保留任务目标、约束与未完成事项，减少关键信息丢失。**执行可恢复性。**增加工具状态与幂等性记录，区分“未执行、已执行、结果未知”，减少重复操作和副作用。**状态持久化效率。**优化状态写入策略，在性能与恢复能力间平衡。以上是可尝试的方向，但是否有效取决于工程和实验结果。